

Cryptanalysis on Asymmetric Structured Key Agreement Schemes

Koki Jimbo*

December 1, 2025

Abstract

We study several asymmetric structured key agreement schemes based on noncommutative matrix operations, including the recent proposal of Lizama as well as the strongly asymmetric algorithms SAA-3 and SAA-5 of Accardi et al. We place them in a common algebraic framework for public key agreement and identify simple structural conditions under which an eavesdropper can reconstruct an effective key-derivation map and reduce key recovery to solving linear systems over finite fields. We then show that the three matrix-based schemes mentioned above all instantiate our algebraic framework and can therefore be broken in polynomial time from public information alone. In particular, their security reduce to the hardness of linear-algebraic problems and does not exceed that of the underlying discrete logarithm problem. Our results demonstrate that the weakness of these schemes is structural rather than parametric, and that minor algebraic modifications are insufficient to repair them.

1 Introduction

In the presence of an eavesdropper Eve, the development of methods that enable Alice and Bob to securely establish a secret shared key(SSK) dates back to the Diffie–Hellman (D–H) key agreement protocol [DH76]. Since then, numerous key agreement schemes have been proposed. Some based on the discrete logarithm problem (DLP) underlying the D–H protocol, and others relying on different computational problems believed to be hard. In recent years, the design of public key agreement(PKA) schemes secure against attacks powered by Shor’s algorithm has attracted considerable interest, with particular focus on lattice-based constructions [DXL12, DGTW19, GdKQ⁺24] and isogeny-based approaches [CLM⁺18]. Protocols aiming to quantum-resistant are not limited to these mainstream, but a variety of other schemes have been developed. Among them, the class most relevant to our work consists of protocols built upon noncommutative algebraic structures.

Examples include the Ko–Lee–Cheon–Han–Kang–Park protocol [KLC⁺00], which uses Artin’s braid groups as a platform and relies on the conjugacy search problem; Stickel’s scheme [Sti05], constructed from multiplication in an arbitrary non-abelian group and often instantiated with matrix groups over finite fields; and schemes based on the triple decomposition problem, such as those proposed by Kurt [Kur06] and Kurt–Peker [Pek14]. A considerable number of noncommutative frameworks have thus been proposed over the past two decades and advertised as quantum-resistant.

However, all of the above-mentioned schemes are now known to be vulnerable to polynomial-time attacks. For instance, Stickel’s matrix-based protocol is broken by Shpilrain’s polynomial-time

*Affiliation: Department of Information Science, Tokyo University of Science.
Email: kjimbo@rs.tus.ac.jp.

linear-algebraic attack [Shp08], which reconstructs the agreed key directly. In the context of braid group-based schemes, representation-theoretic reductions such as Lawrence–Krammer representation [Kra02] enable linear-algebraic cryptanalysis [CJ03] of the protocol [KLC⁺00]. Likewise, the triple decomposition–based schemes [Kur06, Pek14] have been cryptanalyzed in [BZKT18]. Detailed summaries of these attacks can be found in [BZKT18, Rom22].

A notable common phenomenon observed in all these attacks is the following. The attacker does not need to recover the exact private keys chosen by Alice and Bob. It suffices to obtain **candidates** for these private keys by constructing an appropriate system of equations from public data after which solving that system directly yields the SSK. In other words, the attacker only needs to derive a set of equations satisfied by the private keys; any solution of this system will generate the correct SSK. Indeed, in all existing cryptanalyses of noncommutative key agreement schemes, the main effort is placed on reducing the protocol’s algebraic structure to a simpler system, typically a system of linear equations, which, once solved, immediately reveals the SSK.

In this paper, we exploit this phenomenon that recovering candidates for the private keys is already sufficient to derive the SSK to analyze a class of key agreement schemes that we refer to as asymmetric key agreement protocols, which differ from traditional D–H type schemes and have received comparatively little cryptanalytic attention. In such protocols, Alice and Bob do not compute the SSK using symmetric computations. Instead, Bob first generates a public key, after which Alice generates her own public key using Bob’s. This setting naturally fits into the framework [AIRO11] referred to as “Strongly Asymmetric Public Key Agreement (SAPKA)”, which we adopt.

The structure of this paper is as follows. In Section 2, we show that the above phenomenon, namely, that obtaining candidates for private keys suffices to derive the SSK, is not limited to D–H type schemes. Rather, it holds for *general* key agreement protocols in which Alice and Bob independently generate private keys and compute the SSK from exchanged messages and their transcript. When formalized in a set-theoretic manner, this property becomes straightforward to prove. In Section 3, we review the SAPKA framework and present an overview of the attacks developed in the subsequent sections. Section 4 analyzes the asymmetric matrix-based construction of Lizama [LP24], which is derived from a matrix DLP. Section 5 examines two schemes referred to as “strongly asymmetric algorithm (SAA)” which are based on matrix nonlinear problems, namely the SAA-3 protocol [AR15] and the SAA-5 protocol [AIJR19]. For SAA-5, polynomial-time attacks were recently given in [AT24, AT25], whereas this paper is the first to analyse SAA-3 and the matrix protocol of [LP24].

Note that in Section 3 we develop our attacks in an algebraically general setting that simultaneously covers the schemes of [LP24, AR15, AIJR19]. In this abstract framework we describe the relevant operations in the underlying algebraic structure, together with the public messages and key derivation maps, and we derive simple conditions under which Eve can compute the SSK in polynomial time from public information alone. The reason for proceeding via Section 3, rather than analysing [LP24, AR15, AIJR19] directly in an ad hoc fashion, is to show that minor algebraic modifications such as replacing the product by conjugation or switching to a different semigroup or ring do not eliminate the underlying vulnerability: any instantiation satisfying the same structural conditions will admit a similar attack.

Moreover, the paper [LP24] presents two matrix-based key agreement algorithms, one of which is described as an “update” of the other, and the scheme SAA-5 [AIJR19] is published as an updated version of the original proposal in [AR15]. In our formalism, however, each “before/after”

pair fits into the same abstract framework: the schemes of [AR15] and [AIJR19] both instantiate the template of Section 3.3, while the two schemes of [LP24] both fall under the framework of Section 3.2. From this point of view, these updates do not change the essential algebraic structure that enables our attacks, and hence do not provide genuine improvements in terms of resistance to key-recovery attacks.

2 Preliminaries

2.1 Formal description of PKA

We focus on two-party n -round PKAs in which Alice and Bob are not assumed to share any secret information before the key agreement starts. In other words, the protocols we consider begin with Alice and Bob independently choosing their private keys from finite sets specified by the scheme.

Our formulation is based on the following interaction between the two parties. In each round, the players generate fresh private keys and public messages as functions of the transcript consisting of all public information revealed in the previous rounds. After the final round, they compute the SSK also refereed to as session key, which is intended to be equal for both parties and remain unknown to any adversary. This formulation follows the spirit of Maurer’s model of [Mau93]. All public messages are transmitted over an insecure channel, meaning that an eavesdropper Eve can observe them and attempt to use this information to deduce the parties’ private data or the SSK. This interaction can be described algorithmically as follows.

Definition 2.1 (n -round PKA). Fix a security parameter λ and key length κ . An n -round key-agreement protocol Π between parties A and B consists of PPT algorithms

$$\{\text{KeyGen}_A^{(i)}, \text{Send}_A^{(i)}\}_{i=1}^n, \quad \{\text{KeyGen}_B^{(i)}, \text{Send}_B^{(i)}\}_{i=1}^n, \quad \text{Derive}_A, \text{Derive}_B,$$

that interact via a public transcript T as follows.

1. **Round** $i = 1, \dots, n$. Given the current transcript T_{i-1} , each party $P \in \{A, B\}$ samples a private input

$$x_P^{(i)} \leftarrow \text{KeyGen}_P^{(i)}(1^\lambda, T_{i-1}),$$

which may depend on T_{i-1} (transcript-adaptive). Then P outputs a public message

$$m_P^{(i)} \leftarrow \text{Send}_P^{(i)}(x_P^{(1)}, \dots, x_P^{(i)}; T_{i-1}),$$

and the transcript is updated to T_i by appending the round- i messages.

2. **Key derivation.** On the final transcript T_n , the parties output

$$K_A \leftarrow \text{Derive}_A(x_A^{(1)}, \dots, x_A^{(n)}; T_n) \in \{0, 1\}^\kappa, \quad K_B \leftarrow \text{Derive}_B(x_B^{(1)}, \dots, x_B^{(n)}; T_n) \in \{0, 1\}^\kappa.$$

Correctness. For all λ , if both parties follow Π with honestly generated $x_A^{(1)}, \dots, x_A^{(n)}, x_B^{(1)}, \dots, x_B^{(n)}$, then

$$\Pr[K_A = K_B] \geq 1 - \text{negl}(\lambda).$$

As in Definition 2.1, it is now standard in modern cryptography to formalize interactive protocols as collections of PPT algorithms that exchange messages and update a transcript, with each algorithm taking the current transcript as (part of) its input. In the remainder of this work, however, we follow the set theoretic description below.

In the algorithmic formulation above, the protocol and its correctness condition are specified separately, namely, the definition first describes how the parties interact, and then adds an explicit correctness requirement $\Pr[K_A = K_B] \geq 1 - \text{negl}(\lambda)$. In the set-theoretic formulation, by contrast, correctness is implicitly built into the definition of an n -round PKA itself. This will allow us, in particular, to state and prove the “elementary but useful observation” of PKAs that underlies our security analysis in an especially transparent way.

Definition 2.2 (n -Round PKA (set theoretical)). An n -round public key agreement (PKA) protocol for $n \in \mathbb{N}$ is defined by the following components and condition:

- $X_A^{(i)}, X_B^{(i)}$ for $i \in \{1, \dots, n\}$: finite sets representing the private key spaces of Alice and Bob. The subscript indicates the party (A or B), while the superscript denotes the round number in which the key is used.
- $P_A^{(i)}, P_B^{(i)}$ for $i \in \{1, \dots, n\}$: finite sets representing the public key spaces of Alice and Bob, used similarly across n rounds.
- $\mathcal{K}$: a finite set representing the shared secret key (SSK) space.

Suppose there exist subsets

$$S_A^{(i)} \subset X_A^{(i)}, \quad S_B^{(i)} \subset X_B^{(i)} \quad (i \in \{1, \dots, n\})$$

and functions

$$\begin{aligned} f_A^{(1)} &: X_A^{(1)} \rightarrow P_A^{(1)}, \quad f_B^{(1)} : X_B^{(1)} \rightarrow P_B^{(1)}, \\ f_A^{(i)} &: X_A^{(1)} \times \dots \times X_A^{(i)} \times P_B^{(1)} \times \dots \times P_B^{(i-1)} \rightarrow P_A^{(i)}, \quad (2 \leq i \leq n), \\ f_B^{(i)} &: X_B^{(1)} \times \dots \times X_B^{(i)} \times P_A^{(1)} \times \dots \times P_A^{(i-1)} \rightarrow P_B^{(i)}, \quad (2 \leq i \leq n), \\ \kappa_A &: X_A^{(1)} \times \dots \times X_A^{(n)} \times P_B^{(1)} \times \dots \times P_B^{(n)} \rightarrow \mathcal{K}, \\ \kappa_B &: X_B^{(1)} \times \dots \times X_B^{(n)} \times P_A^{(1)} \times \dots \times P_A^{(n)} \rightarrow \mathcal{K}, \end{aligned}$$

such that for all

$$(x_A^{(1)}, \dots, x_A^{(n)}) \in S_A^{(1)} \times \dots \times S_A^{(n)}, \quad (x_B^{(1)}, \dots, x_B^{(n)}) \in S_B^{(1)} \times \dots \times S_B^{(n)},$$

if we define recursively

$$\begin{aligned} y_A^{(1)} &:= f_A^{(1)}(x_A^{(1)}), \quad y_B^{(1)} := f_B^{(1)}(x_B^{(1)}), \\ y_A^{(i)} &:= f_A^{(i)}(x_A^{(1)}, \dots, x_A^{(i)}, y_B^{(1)}, \dots, y_B^{(i-1)}) \quad (2 \leq i \leq n), \\ y_B^{(i)} &:= f_B^{(i)}(x_B^{(1)}, \dots, x_B^{(i)}, y_A^{(1)}, \dots, y_A^{(i-1)}) \quad (2 \leq i \leq n), \end{aligned}$$

then the following must hold:

$$\kappa_A(x_A^{(1)}, \dots, x_A^{(n)}, y_B^{(1)}, \dots, y_B^{(n)}) = \kappa_B(x_B^{(1)}, \dots, x_B^{(n)}, y_A^{(1)}, \dots, y_A^{(n)}). \quad (2.1)$$

In this case, the tuple

$$(f_A^{(1)}, \dots, f_A^{(n)}, f_B^{(1)}, \dots, f_B^{(n)}, \kappa_A, \kappa_B)$$

is called an n -round key agreement protocol defined over the subsets $S_A^{(1)} \times \dots \times S_A^{(n)}$ and $S_B^{(1)} \times \dots \times S_B^{(n)}$.

The above mentioned “elementary but useful observation” on PKAs is explained and derived automatically from this definition as follows.

Proposition 2.3. Let $X_A^{(i)}, X_B^{(i)}, P_A^{(i)}, P_B^{(i)}$ be given sets for $i \in \{1, \dots, n\}$, and let

$$(f_A^{(1)}, \dots, f_A^{(n)}, f_B^{(1)}, \dots, f_B^{(n)}, \kappa_A, \kappa_B)$$

be an n -round PKA protocol defined over subsets

$$S_A := S_A^{(1)} \times \dots \times S_A^{(n)} \subset X_A^{(1)} \times \dots \times X_A^{(n)}, \quad S_B := S_B^{(1)} \times \dots \times S_B^{(n)} \subset X_B^{(1)} \times \dots \times X_B^{(n)}.$$

Suppose $(x_A^{(1)}, \dots, x_A^{(n)}) \in S_A$ and $(x_B^{(1)}, \dots, x_B^{(n)}) \in S_B$ are chosen uniformly at random, and define the public messages inductively as:

$$\begin{aligned} y_A^{(1)} &:= f_A^{(1)}(x_A^{(1)}), & y_B^{(1)} &:= f_B^{(1)}(x_B^{(1)}), \\ y_A^{(i)} &:= f_A^{(i)}(x_A^{(1)}, \dots, x_A^{(i)}, y_B^{(1)}, \dots, y_B^{(i-1)}) & \text{for } i = 2, \dots, n, \\ y_B^{(i)} &:= f_B^{(i)}(x_B^{(1)}, \dots, x_B^{(i)}, y_A^{(1)}, \dots, y_A^{(i-1)}) & \text{for } i = 2, \dots, n. \end{aligned}$$

Then any tuple $(x_E^{(1)}, \dots, x_E^{(n)})$ satisfying either all of the following:

- $x_E^{(1)} \in \{x \in X_A^{(1)} \mid f_A^{(1)}(x) = y_A^{(1)}\}$,
- For all $i = 2, \dots, n$:
 $(x_E^{(1)}, \dots, x_E^{(i)}) \in \{(x_1, \dots, x_i) \in X_A^{(1)} \times \dots \times X_A^{(i)} \mid f_A^{(i)}(x_1, \dots, x_i, y_B^{(1)}, \dots, y_B^{(i-1)}) = y_A^{(i)}\}$,
- $(x_E^{(1)}, \dots, x_E^{(n)}) \in S_A$

or all of the corresponding conditions for Bob (with A and B swapped), also satisfies

$$\kappa_A(x_E^{(1)}, \dots, x_E^{(n)}, y_B^{(1)}, \dots, y_B^{(n)}) = \kappa \quad \text{or} \quad \kappa_B(x_E^{(1)}, \dots, x_E^{(n)}, y_A^{(1)}, \dots, y_A^{(n)}) = \kappa,$$

respectively.

Proof. We prove the case where $(x_E^{(1)}, \dots, x_E^{(n)})$ satisfies the three conditions for Alice; the proof for Bob is symmetric.

Since $x_E^{(1)}$ satisfies the first condition and $(x_E^{(1)}, \dots, x_E^{(i)})$ satisfies the second condition for all i , we have

$$\begin{aligned} f_A^{(1)}(x_E^{(1)}) &= y_A^{(1)}, \\ f_A^{(i)}(x_E^{(1)}, \dots, x_E^{(i)}, y_B^{(1)}, \dots, y_B^{(i-1)}) &= y_A^{(i)} \quad \text{for all } i = 2, \dots, n. \end{aligned}$$

Together with the third condition,

$$\kappa_A(x_E^{(1)}, \dots, x_E^{(n)}, y_B^{(1)}, \dots, y_B^{(n)}) = \kappa_B(x_B^{(1)}, \dots, x_B^{(n)}, y_A^{(1)}, \dots, y_A^{(n)}) = \kappa$$

by the definition of the PKA protocol. □

Proposition 2.3 supports the general point that deriving the SSK in a key-agreement protocol does not necessarily require recovering either party’s private key.

The role of the sets S^A and S^B are restriction for Eve which imposes precisely to make preimage search for the public map hard in the sense of Proposition 2.3. Concretely, in [DXL12, DGTW19, GdKQ⁺24], a public vector $y \equiv Ax + e \pmod{q}$ is given and S^A corresponds to a set of “small” vectors; the hardness assumption is that recovering $(x, e) \in S^A \times S^A$ from (A, y) is computationally infeasible, even though finding *some* preimage $(x, e) \in (\mathbb{Z}/q\mathbb{Z})^n \times (\mathbb{Z}/q\mathbb{Z})^m$ is easy if smallness is not required. In Roman’kov’s improvement of the noncommutative Diffie–Hellman protocol [Rom22], the role of S^A, S^B is played by “marginal sets”: given a publicly derived subset C_π , it is intended to be computationally hard to construct a group word u for which C_π itself is marginal, or for which the marginal set associated to u contains C_π .

In PKAs where S_A or S_B does not properly restrict $X_A X_B$, or where $S_A = X_A$ (resp. $S_B = X_B$), by Proposition 2.3, if obtaining a preimage is easy, the SSK can be easily derived. As stated above, there are, perhaps unexpectedly, many PKAs claiming to be post-quantum (PQC) that overlook this viewpoint, and we will see this concretely in Section 4 and 5.

One possible line of defence against our attacks on [AR15, AIJR19, LP24] in Section 4 and 5 would be to identify appropriate subsets S^A, S^B that impose analogous restrictions as [DXL12, DGTW19, Rom22, GdKQ⁺24] and thereby render the corresponding preimage search intractable. Investigating whether such restricted domains can be defined and instantiated in a secure and practical way is a natural direction for countermeasures, but we do not pursue this topic further in this paper.

Remark 2.4. In the noise free channel setting, the impossibility of information theoretical PKA is automatically derived from the proposition 2.3. Since $S_A^{(i)}$ and $S_B^{(i)}$ are public (as otherwise, Alice and Bob would not be able to generate the shared secret key with arbitrary probability), Eve can verify whether any element $x_E^{(i)}$ obtained from the set

$$\left\{ x \in X_A^{(i)} \mid f_A^{(i)}(x, y_B^{(1)}, y_B^{(2)}, \dots, y_B^{(i-1)}) = y_A^{(i)} \right\}$$

belongs to $S_A^{(i)}$, assuming she has unlimited computational resources and storage capacity. This is because $S_A^{(i)}$ is a finite set, and Eve can simply enumerate all of its elements and perform equality checks to determine whether $x_E^{(i)} \in S_A^{(i)}$. Thus, by repeating this check for each round until she finds an $x_E \in S_A$ (or S_B), Eve can eventually identify the correct value of the shared key κ with probability 1.

This fact was already established in [Mau93] from the viewpoint of secret-key rate and is well known. Proposition 2.3 can be viewed as a specialised reformulation of this phenomenon within our abstract PKA framework.

3 Overview of attack against asymmetric public key agreement

As noted above, we call a PKA *asymmetric* if Alice and Bob may use different private/public key spaces and follow different key-generation rules. Among these, we focus on schemes in which the two parties’ public values are not produced simultaneously: one party’s public value is computed using the other party’s public map. We assume that Bob first publishes the relevant public maps, and then Alice uses them to produce her public value. This setting is formalized in [AIRO11] and we recast it here with notation consistent with Definition 2.2.

- $X_A^{(1)}$: Alice's private-key space;
- $S_A^{(1)} \subseteq X_A^{(1)}$: the protocol-admissible subset (constraint set);
- $P_A^{(1)}$: Alice's public-key space;
- $\mathcal{K}$: the shared-secret key (SSK) space;
- maps $f_A : S_A^{(1)} \rightarrow P_A^{(1)}$, $k_A : S_A^{(1)} \rightarrow \mathcal{K}$, and $k_B : Y_A^{(1)} \rightarrow \mathcal{K}$,

which satisfy

$$k_A(x) = (k_B \circ f_A)(x) \quad \text{for all } x \in S_A^{(1)}. \quad (3.1)$$

Based on these components, the key-agreement proceeds as follows.

- **Bob key generation:** Bob specifies maps $f_A : S_A^{(1)} \rightarrow P_A^{(1)}$ and $k_A : S_A^{(1)} \rightarrow \mathcal{K}$ (to be public), and $k_B : P_A^{(1)} \rightarrow \mathcal{K}$ (kept secret), such that (3.1) holds.
- **Alice key generation:** Alice samples $x_A \in S_A^{(1)}$ uniformly at random as her secret key, computes $y_A := f_A(x_A) \in P_A^{(1)}$ as her public key, and sends y_A to Bob.
- **SSK generation:** Alice computes $K_A := k_A(x_A) \in \mathcal{K}$; Bob computes $K_B := k_B(y_A) \in \mathcal{K}$. By (3.1), $K_B = K_A$.

Here, a map is *public* in the algorithmic sense: e.g., f_A is public if there exists a PPT algorithm A and public auxiliary input $\mathbf{aux}$ such that for every $x \in S_A^{(1)}$, $A(x, \mathbf{aux}) = f_A(x)$ (and the description of A is efficiently constructible). The same applies to k_A .

From this perspective, we can regard the above asymmetric type PKAs as algorithm composed of the following ingredients: As in Definition 2.1, we formalize them as follows.

Definition 3.1 (Asymmetric-Type PKA). Fix a security parameter λ and a key length κ . A two-party *asymmetric* key agreement protocol Π between parties A (Alice) and B (Bob) consists of PPT algorithms $\text{KeyGen}_A, \text{KeyGen}_B, \text{Send}_1^B, \text{Send}_2^B, \text{Send}^A, \text{Derive}_A, \text{Derive}_B$ such that:

1. **Private-key generation.** Alice samples a private value $\text{sk}_A \leftarrow \text{KeyGen}_A(1^\lambda)$, and Bob samples $(\text{sk}_B^1, \text{sk}_B^2) \leftarrow \text{KeyGen}_B(1^\lambda)$.
2. **Message exchange.** Bob first computes and sends to Alice $\text{m}_B^1 \leftarrow \text{Send}_1^B(\text{sk}_B^1; \perp)$ and $\text{m}_B^2 \leftarrow \text{Send}_2^B(\text{sk}_B^2; \perp)$, where $\perp$ denotes the empty input. Upon receiving $(\text{pk}_B^1, \text{pk}_B^2)$, Alice computes $\text{m}_A \leftarrow \text{Send}^A(\text{sk}_A; \text{pk}_B^1)$ and sends pk_A to Bob.
3. **Key derivation.** Finally, the parties output

$$K_A \leftarrow \text{Derive}_A(\text{sk}_A; \text{pk}_B^2) \in \{0, 1\}^\kappa, \quad K_B \leftarrow \text{Derive}_B(\text{sk}_B^1, \text{sk}_B^2; \text{pk}_A) \in \{0, 1\}^\kappa.$$

Correctness. With honestly generated secrets and honest execution, the protocol is correct if $\Pr[K_A = K_B] \geq 1 - \text{negl}(\lambda)$.

We consider only the case where Alice's public value depends on Bob's initial public value pk_B^1 : Alice can compute her SSK K_A at the time she forms pk_A from pk_B^1 , while Bob computes K_B as soon as he receives pk_A (no further message from Bob is required after Alice's reply).

3.1 Cryptanalysis against the asymmetric PKAs

As in Proposition 2.3, attacks on the above PKA can be broadly divided into two kinds: from Alice’s side and from Bob’s side. Our strategy, following Proposition 2.3, is to recover the SSK by feeding a suitable *candidate* private input into the *function* that derives the SSK. On Bob’s side, this means that our goal is to recover the SSK-generation map k_B itself. However, in [AR15, AIJR19, LP24] the map k_B is not given explicitly: Bob’s computation of the SSK is described as some procedure that is not presented in the form of the private-key-based map (3.1). Nevertheless, because of the simple structure of these schemes—in terms of the underlying semigroup operation in [LP24], and in terms of ring operations in [AR15, AIJR19]—one can reconstruct an appropriate k_B from the key-generation procedures and the public messages of Alice and Bob. In both cases, the SSK can be recovered without reconstructing Bob’s entire private key.

We begin with the attack from Alice’s side. It simply consists of searching for a candidate private key that satisfies the defining relation between Alice’s public value and her public key generation map.

Attack from Alice’s side. Since $y_A = f_A(x_A)$ is public, if Eve can obtain $x_E \in S_A^{(1)}$ such that $y_A = f_A(x_E)$, then by inputting x_E to k_A she gets

$$k_A(x_E) = (k_B \circ f_A)(x_E) = k_B(f_A(x_E)) = k_B(y_A) = k_B(f_A(x_A)) = K_A,$$

and thus derives the SSK.

Attack from Bob’s side. Construct a map k_E satisfying

$$k_A(x) = (k_E \circ f_A)(x) \quad \text{for all } x \in S_A^{(1)}.$$

Then Eve outputs $k_E(y_A)$, which satisfies:

$$k_E(y_A) = k_E(f_A(x_A)) = k_A(x_A) = K_A$$

. Note that, unlike k_B , the map k_E does not need to be defined on the entire set $Y_A^{(1)}$. We therefore distinguish between the two and assume that Eve focuses on deriving such $k_E : \text{Im}(f_A) \rightarrow \mathcal{K}$.

At a high level, the schemes of [LP24] and [AR15, AIJR19] fit into the abstract frameworks described in Sections 3.2 and 3.3 below, respectively. In what follows, we will show that, in both cases, the map k_E is in fact uniquely determined in terms of the underlying semigroup or ring operations. We initially treat k_B as unknown.

When k_A is sent, as noted above, Bob publishes some message $m^{(1)}$, and Alice computes k_A from that message together with her private key x_A ; that is, there exists a polynomial-time computable function

$$\widehat{k}_A : S_A^{(1)} \times Y_B^{(1)} \rightarrow \mathcal{K}$$

such that $k_A(x) = \widehat{k}_A(x, m^{(1)})$ for $m^{(1)} \in Y_B^{(1)}$. Similarly, for y_A there exists a polynomial-time computable function

$$\widehat{f}_A : S^{(1)} \times Y_B^{(2)} \rightarrow S^{(2)}$$

such that $f_A(x) = \widehat{f}_A(x, m^{(2)})$ for $m^{(2)} \in Y_B^{(2)}$. Here $Y_B^{(1)}$ and $Y_B^{(2)}$ are Bob’s public key (message) space as Definition 2.2. Let the functions for generating $m^{(1)}, m^{(2)}$ be $f_B^{(1)} : X_B^{(1)} \rightarrow Y_B^{(1)}$ and

$f_B^{(2)} : X_B^{(2)} \rightarrow Y_B^{(2)}$, and write $m^{(1)} = f_B^{(1)}(x_B^{(1)})$, $m^{(2)} = f_B^{(2)}(x_B^{(2)})$. Then, by (3.1), the function k_E that Eve seeks must satisfy

$$\widehat{k_A}(x, f_B^{(1)}(x_B^{(1)})) = k_E \circ \widehat{f_A}(x, f_B^{(2)}(x_B^{(2)})). \quad (3.2)$$

Since the algorithms that evaluate $\widehat{k_A}$ and $\widehat{f_A}$ are implemented on Alice's device prior to the start of key agreement, note that (3.2) must either hold for every choice of $x_B^{(1)} \in S_B^{(1)} \subseteq X_B^{(1)}$ and $x_B^{(2)} \in S_B^{(2)} \subseteq X_B^{(2)}$, or else the public messages $f_B^{(1)}(x_B^{(1)})$ and $f_B^{(2)}(x_B^{(2)})$ are fixed in advance where $S_B^{(i)}$ are corresponding admissible private key set for Bob. In the latter case, the message values do not depend on Bob's private choices $x_B^{(i)}$. Writing $\widehat{K} := \widehat{k_A}(\cdot, m^{(1)})$ and $\widehat{F} := \widehat{f_A}(\cdot, m^{(2)})$, the condition on k_E that Eve must meet then reduces simply to $\widehat{K} = k_E \circ \widehat{F}$. We do not have more to say in this case.

3.2 Attack against [LP24] Type

Setting. Let $X_A^{(1)} = S_A^{(1)} = \mathbb{N}$ and

$$X_B^{(1)} = X_B^{(2)} = S_B^{(1)} = S_B^{(2)} = Y_B^{(1)} = Y_B^{(2)} = Y_A^{(1)} =: S,$$

where $(S, *)$ is a semigroup with identity e_S . For $a \in S$ and $x \in \mathbb{N}$, write

$$a^x := \underbrace{a * a * \cdots * a}_{x \text{ times}}, \quad a^0 := e_S.$$

Let $*_L$ be a left action of S on itself. We assume the following compatibility between the left action and powers:

$$v *_L (a^x) = (v *_L a)^x \quad (\forall v, a \in S, \forall x \in \mathbb{N}). \quad (3.3)$$

Introduce the family of maps $\{g_u : S \rightarrow S\}_{u \in \mathbb{N}}$ by

$$g_u(a) := a^u. \quad (3.4)$$

Alice chooses an exponent $x \in \mathbb{N}$ and uses

$$f_A(x, z) := g_x(z) = z^x, \quad k_A(x, y) := g_x(y) = y^x. \quad (3.5)$$

Bob chooses (independently) an exponent $y \in \mathbb{N}$ and secrets $x_B^{(1)}, x_B^{(2)} \in S$, and publishes the messages

$$m^{(1)} := g_y(x_B^{(1)}) = (x_B^{(1)})^y, \quad m^{(2)} := g_y(x_B^{(2)}) = (x_B^{(2)})^y. \quad (3.6)$$

Construction of k_E . Assume there exists $v \in S$ such that

$$v *_L m^{(2)} = m^{(1)}. \quad (3.7)$$

And Eve solves this equation for $v \in S$. Define k_E on the image of $f_A(\cdot, m^{(2)})$ denoted by $\Gamma := \{f_A(x, m^{(2)}) = (m^{(2)})^x : x \in \mathbb{N}\}$ as

$$k_E(f_A(x, m^{(2)})) := v *_L f_A(x, m^{(2)}) = v *_L ((m^{(2)})^x). \quad (3.8)$$

Then, for every $x \in \mathbb{N}$,

$$\begin{aligned}
k_E \circ f_A(x, m^{(2)}) &= v *_L ((m^{(2)})^x) && \text{by (3.8)} \\
&= (v *_L m^{(2)})^x && \text{by (3.3)} \\
&= (m^{(1)})^x && \text{by (3.7)} \\
&= g_x(m^{(1)}) = k_A(x, m^{(1)}).
\end{aligned}$$

Here, k_E is uniquely determined on Γ . In particular, if $f_A(\cdot, m^{(2)})$ is surjective onto S (so $\Gamma = S$), then any $k_B : S \rightarrow S$ satisfying $k_B \circ f_A(\cdot, m^{(2)}) = k_A(\cdot, m^{(1)})$ must agree with k_E on S ; hence $k_B = k_E$.

Assuming that, for all $v, a \in S$, the left action $v *_L a$ can be evaluated in polynomial time in the parameters of the scheme, we can summarise the above discussion in the following theorem.

Theorem 3.2. *In the above setting, let $m^{(1)}, m^{(2)} \in S$ be Bob's public messages and let $y_A = f_A(x_A, m^{(2)}) \in \Gamma$ be Alice's public value. Suppose that Eve can efficiently find an element $v \in S$ such that*

$$v *_L m^{(2)} = m^{(1)}.$$

Then Eve can recover the corresponding shared secret key in polynomial time from $(m^{(1)}, m^{(2)}, y_A)$.

3.3 Attack against Strongly Asymmetric Algorithms (SAAs)

As in the group-action setting, SAA schemes [AR15, AIJR19] can be attacked without recovering k_B or Bob's secrets $x_B^{(1)}, x_B^{(2)}$. Below is the high-level template used in Section 5.

Setting. Let R be a (possibly noncommutative) ring and put $V := R^n$ with its natural R - R bimodule structure (componentwise left/right multiplication). Bob's private keys are chosen from this set so we set $X_B^{(1)} = X_B^{(2)} = S_B^{(1)} = S_B^{(2)} = V$. Let the Alice's private key space $S_A^{(1)}$ be a subset of V which is specified in the protocols [AR15] (In SAA5 [AIJR19] $S_A^{(1)} = V$). Let $B : V \times V \rightarrow R$ be (partially) R -bilinear in both variables in the sense:

$$rB(x, y) = B(x, ry) \quad (\forall x, y \in V, \forall r \in R). \quad (3.9)$$

and for some limited subset $T \subset R$, we assume for all private key of Alice $x \in S_A^{(1)}$ and $v \in T$, the following holds:

$$B(x, yv) = B(x, y)v \quad (\forall y \in V). \quad (3.10)$$

Let $g : R \rightarrow R$ and define the componentwise map $g' : V \rightarrow V$ by

$$g'(y_1, \dots, y_n) := (g(y_1), \dots, g(y_n)). \quad (3.11)$$

where g and g' have the following kernel-congruence compatibilities:

$$g(r) = g(r') \Rightarrow g(rv) = g(r'v) \quad (\forall v \in R). \quad (3.12)$$

$$g(r) = g(r') \Rightarrow g(ur) = g(ur') \quad (\forall u \in R). \quad (3.13)$$

$$g'(y) = g'(y') \Rightarrow g(B(x, y)) = g(B(x, y')) \quad (\forall x \in V, \forall y, y' \in V). \quad (3.14)$$

We treat the second input of the following maps as a public *tag* $y \in \text{Im}(g')$ rather than a raw vector in V . Given $y \in \text{Im}(g')$, pick any $z \in V$ with $g'(z) = y$ and define

$$f_A(x, y) := g(B(x, z)), \quad k_A(x, y) := g(B(x, z)).$$

By the kernel-congruence assumption for g' , these values do not depend on the chosen preimage z , so $f_A, k_A : V \times \text{Im}(g') \rightarrow \text{Im}(g)$ are well-defined.

Construction of k_E Let $m^{(1)} := g'(x_B^{(1)})$ and $m^{(2)} := g'(x_B^{(2)})$ be Bob's public messages. Pick any preimages $t, s \in V$ with $g'(t) = m^{(1)}$ and $g'(s) = m^{(2)}$, and suppose there exists $u, v \in R$ such that

$$u s v = t \quad (\text{componentwise in } V = R^n). \quad (3.15)$$

$$v \in T \quad (3.16)$$

Define the subset

$$\Gamma := \left\{ f_A(x, m^{(2)}) = g(B(x, x_B^{(2)})) : x \in V \right\} \subseteq \text{Im}(g).$$

Define k_E on Γ by

$$k_E(f_A(x, m^{(2)})) := g(u B(x, s) v) \quad (\forall x \in V). \quad (3.17)$$

Here, k_E is well-defined on Γ . Let $s, s' \in V$ be any preimages of $m^{(2)}$, i.e. $g'(s) = g'(s') = m^{(2)}$. By the congruence (3.14), we have

$$g(B(x, s)) = g(B(x, s')) \quad (\forall x \in V).$$

Applying the left and right-multiplication congruence (3.13) and (3.12) with $r = B(x, s)$ and $r' = B(x, s')$ then yields

$$g(urv) = g(ur'v).$$

Hence the right-hand side of (3.17) is independent of the choice of the representative s with $g'(s) = m^{(2)}$, and k_E is well-defined on Γ .

For all $x \in V$, the following relation holds.

$$\begin{aligned} k_E \circ f_A(x, m^{(2)}) &= g(u B(x, s) v) && \text{by (3.17) (with } g'(s) = m^{(2)}) \\ &= g(B(x, usv)) && \text{by } R\text{-balance (3.9)(3.10)} \\ &= g(B(x, t)) && \text{by (3.15)} \\ &= g(B(x, x_B^{(1)})) && \text{by (3.14) since } g'(t) = g'(x_B^{(1)}) \\ &= k_A(x, m^{(1)}) && \end{aligned} \quad (3.18)$$

Assuming that the ring and module operations in R and V , the bilinear map B , and the maps g, g' can all be evaluated in polynomial time, we may summarise the above discussion as follows.

Theorem 3.3. *In the setting of Section 3.3, let $m^{(1)} := g'(x_B^{(1)})$ and $m^{(2)} := g'(x_B^{(2)})$ be Bob's public messages in $\text{Im}(g')$, and let $y_A := f_A(x_A, m^{(2)}) \in \Gamma$ be Alice's public value for some private key $x_A \in S_A^{(1)}$. Suppose that Eve can efficiently find elements $u \in R$, $v \in T$ and preimages $t, s \in V$ with $g'(t) = m^{(1)}$ and $g'(s) = m^{(2)}$ such that*

$$usv = t \quad \text{componentwise in } V = R^n.$$

Then, defining k_E on Γ as in (3.17), Eve can compute the corresponding SSK as $k_E(y_A)$ in polynomial time from the public transcript $(m^{(1)}, m^{(2)}, y_A)$.

Remark 3.4. In the algorithmic viewpoint of Definition 3.1, once Bob's private keys and public messages are fixed, his key-derivation procedure induces a map k_B on the set Γ of Alice's reachable public values. The identities of Sections 3.2 and 3.3 show that the map k_E constructed above is the *unique* map $k : \Gamma \rightarrow \mathcal{K}$ satisfying

$$k \circ f_A(\cdot, m^{(2)}) = k_A(\cdot, m^{(1)}),$$

and hence $k_B|_\Gamma = k_E$. In particular, whenever Bob can compute the SSK in polynomial time, the same is true for an eavesdropper: evaluating the explicitly reconstructed map k_E on $y_A \in \Gamma$ yields the SSK in polynomial time from the public transcript.

4 Attack against [LP24]

In [LP24], two PKA schemes are proposed, both built on matrix multiplication over $\mathbb{F}_p$ together with integer powers of matrices. We begin with the first algorithm.

4.1 PKAs of [LP24]

First PKA algorithm

Public parameters Fix $n \in \mathbb{N}$ and a prime p . Let $S := M(n, \mathbb{F}_p)$ be set of $n \times n$ matrices of $\mathbb{F}_p$ entries and we assume Alice and Bob agree with this set a priori.

The protocol.

Bob key generation: Let $w, z \in \text{GL}_n(\mathbb{F}_p)$ be Bob's private matrices (chosen so that $wz \neq zw$).

Bob publishes

$$m^{(1)} := zw \quad \text{and also} \quad m^{(2)} := wz.$$

Bob sends both to Alice.

Alice key generation: Alice samples a random exponent $x \in \mathbb{N}$ and returns:

$$y_A := (m^{(2)})^x = (wz)^x$$

to Bob.

The SSK generation Alice uses another public message $m^{(1)}$ from Bob to calculate K_A by:

$$K_A := (m^{(1)})^x = (zw)^x$$

Bob computes the SSK K_B using y_A from Alice as:

$$K_B := w^{-1}y_A w = w^{-1}(wz)^x w = (zw)^x = K_A$$

Remark 4.1. The authors note two weaknesses of the first protocol: (i) if wz is diagonalizable, the problem reduces to independent discrete logarithms, and (ii) the raw products zw and wz yield $2n^2$ equations in $2n^2$ unknowns, enabling linear recovery of w and z . To address these, the second approach never publishes the *raw* products; instead, each party publishes *powered* versions $(zw)^y$ and $(wz)^y$ under a secret exponent y , and the exchange uses only these masked values. This modification is intended to hinder both diagonalization-based reductions and direct linear factorization attacks on (w, z) .

Second PKA algorithm

Public parameters Fix $n \in \mathbb{N}$ and a prime p . Let $S := M(n, \mathbb{F}_p)$ be the set of $n \times n$ matrices over $\mathbb{F}_p$.

The protocol.

Bob key generation: Bob samples private matrices $w, z \in \text{GL}_n(\mathbb{F}_p)$ and a private exponent $y \in \mathbb{N}$. He publishes the pair of powered public keys

$$m^{(1)} := (zw)^y, \quad m^{(2)} := (wz)^y.$$

Alice key generation: Upon receiving $m^{(2)}$, Alice samples an exponent $x \in \mathbb{N}$ and computes

$$y_A := (m^{(2)})^x = (wz)^{xy}.$$

She sends y_A to Bob.

The SSK generation: Alice derives

$$K_A := (m^{(1)})^x = (zw)^{xy}.$$

Using y_A , Bob computes

$$K_B := w^{-1}y_A w = w^{-1}(wz)^{xy} w = (zw)^{xy} = K_A.$$

Thus both parties obtain the same session key.

4.2 The Attacks

As noted in Remark 4.1, the first algorithm can be broken in polynomial time by recovering Bob's private matrices. The second algorithm is intended to mitigate this weakness by publishing only exponentiated products, thereby masking the raw factors wz and zw . The authors claim that, under the hardness of the matrix DLP, namely, given M and M^x for a randomly sampled $x \in \mathbb{N}$, recovering x is computationally infeasible, the second protocol is secure at least against SSK recovery from public information.

While this assumption plausibly protects Alice's secret exponents, we argue that Bob's private parameters remain vulnerable to a Bob-side attack as described in Section 3.1. In particular, Eve can reconstruct the *implicitly* defined map k_B and thereby compute the SSK. Specifically, we show the following as a corollary of Theorem 3.2:

Corollary 4.2. *There is deterministic polynomial time algorithm to calculate the SSK of the PKA schemes of [LP24] only from public information.*

Following Section 3.1, we fix:

- $S_A^{(1)} = \mathbb{N}$, $S = \text{GL}_n(\mathbb{F}_p)$,
- the left action $*_L$ on S is conjugation: $a *_L m := a^{-1}ma$,
- the maps a^u and g_u are as in (3.4) (for the *first* algorithm one may take $g_u = \text{id}$ for all $u \in \mathbb{N}$),
- Bob's (derived) secrets are $x_B^{(1)} := zw$ and $x_B^{(2)} := wz$,
- $f_A(x, z)$ and $k_A(x, z)$ are defined as in (3.5).

The conjugation action is compatible with taking powers, namely,

$$v *_L (a^x) = v^{-1}a^xv = (v^{-1}av)^x = (v *_L a)^x \quad (\forall v, a \in S, \forall x \in \mathbb{N}), \quad (4.1)$$

which is the instance of (3.3) for conjugation. For the second algorithm, with Bob's private $y \in \mathbb{N}$, his public values are (cf. (3.6))

$$m^{(1)} = g_y(x_B^{(1)}) = (zw)^y, \quad m^{(2)} = g_y(x_B^{(2)}) = (wz)^y. \quad (4.2)$$

(For the *first* algorithm, take $y = 1$, i.e. $m^{(1)} = zw$, $m^{(2)} = wz$.)

By the general recipe in Section 3.1, Eve solves the equation

$$v *_L m^{(2)} = m^{(1)} \iff v^{-1}(wz)^y v = (zw)^y \iff (wz)^y v = v(zw)^y, \quad (4.3)$$

for $v \in S$. This is a linear system over $\mathbb{F}_p$ in the n^2 unknown entries of v (hence solvable in polynomial time); moreover, $v = w$ is a witness solution since $w^{-1}(wz)^y w = (zw)^y$.

With any solution v , define k_E as in (3.8):

$$k_E(f_A(x, m^{(2)})) := v^{-1}(m^{(2)})^x v \quad (\forall x \in \mathbb{N}). \quad (4.4)$$

Then, for every $x \in \mathbb{N}$,

$$\begin{aligned}
k_E \circ f_A(x, m^{(2)}) &= v^{-1}((m^{(2)})^x)v = v^{-1}(((wz)^y)^x)v \\
&= (v^{-1}(wz)^yv)^x && \text{by (4.1)} \\
&= ((zw)^y)^x && \text{by (4.3)} \\
&= g_x(m^{(1)}) = k_A(x, m^{(1)}).
\end{aligned}$$

Therefore, with this k_E and Alice's public value $y_A = f_A(x, m^{(2)}) = (m^{(2)})^x$, Eve computes the SSK. Since (4.3) reduces to a linear system over $\mathbb{F}_p$, v (and hence the SSK) is obtainable in polynomial time.

5 Attack against Strongly Asymmetric Algorithm (SAA)s

5.1 SAA3

The SAA3 key-agreement scheme is built on an entrywise (Schur) exponentiation of matrices. Fix a prime p and an integer $d \geq 1$. For $a \in \mathbb{F}_p$ and $M = (M_{i,j}) \in M(d, \mathbb{F}_p)$, define the Schur-exponentiation

$$a^{\circ M} := (a^{M_{i,j}})_{1 \leq i, j \leq d} \in M(d, \mathbb{F}_p),$$

where exponents are taken in $\mathbb{Z}/(p-1)\mathbb{Z}$ (so $a^{u+v} = a^u a^v$ and $(a^u)^v = a^{uv}$ for $a \in \mathbb{F}_p^\times$).¹

Define, for $M \in M(d, \mathbb{F}_p)$, left and right “action” maps $L_M, R_M : M(d, \mathbb{Z}/(p-1)\mathbb{Z}) \rightarrow M(d, \mathbb{F}_p)$ by

$$(L_M(X))_{i,j} := \prod_{k=1}^d M_{k,j}^{X_{i,k}}, \quad (5.1)$$

$$(R_M(X))_{i,j} := \prod_{k=1}^d M_{i,k}^{X_{k,j}}, \quad (1 \leq i, j \leq d). \quad (5.2)$$

Equivalently, in block form,

$$L_M(X) = \begin{pmatrix} \prod_k M_{k,1}^{X_{1,k}} & \cdots & \prod_k M_{k,d}^{X_{1,k}} \\ \vdots & & \vdots \\ \prod_k M_{k,1}^{X_{d,k}} & \cdots & \prod_k M_{k,d}^{X_{d,k}} \end{pmatrix}, \quad R_M(X) = \begin{pmatrix} \prod_k M_{1,k}^{X_{k,1}} & \cdots & \prod_k M_{1,k}^{X_{k,d}} \\ \vdots & & \vdots \\ \prod_k M_{d,k}^{X_{k,1}} & \cdots & \prod_k M_{d,k}^{X_{k,d}} \end{pmatrix}.$$

With this notation, for all $a \in \mathbb{F}_p$ and $X, Y \in M_d(\mathbb{Z}/(p-1)\mathbb{Z})$ one has

$$L_{a^{\circ X}}(Y) = a^{\circ(YX)}, \quad (5.3)$$

$$R_{a^{\circ X}}(Y) = a^{\circ(XY)}. \quad (5.4)$$

Indeed, for (5.3) the (i, j) -entry is $\prod_k (a^{X_{k,j}})^{Y_{i,k}} = a^{\sum_k Y_{i,k} X_{k,j}} = a^{(YX)_{i,j}}$, and similarly for (5.4). The protocol of SAA3 is as follows.

¹As usual, exponents are interpreted modulo $p-1$ for nonzero base; one may restrict to $M(d, \mathbb{F}_p^\times)$ if desired.

Public parameters Fix $d \in \mathbb{N}$ and a prime p . Let $R := M(d, \mathbb{F}_p)$ be the ring of $d \times d$ matrices over $\mathbb{F}_p$. Fix $\alpha \in R$ and a finite index set $I = \{i_1, \dots, i_n\} \subset \mathbb{N}$, and view $V := R^n$ as an R -bimodule.

Bob key generation Bob samples private $x_B \in T \subseteq R$, $N_B \in \text{GL}_d(\mathbb{F}_p) \subset R$, a private scalar $c \in \mathbb{F}_p^\times$, and a private n -tuple $(B_{i_1}, \dots, B_{i_n}) \in V$, where T is a set defined by:

$$T = \{v \in R \mid [x_B, \alpha] := x_B \alpha - \alpha x_B = 0\}. \quad (5.5)$$

He publishes two n -tuples $(m_{i_j}^{(1)})_{j=1}^n$ and $(m_{i_j}^{(2)})_{j=1}^n$ defined by

$$m_{i_j}^{(1)} := c^{\circ(B_{i_j} x_B)}, \quad m_{i_j}^{(2)} := c^{\circ(N_B B_{i_j})} \quad (j = 1, \dots, n).$$

Alice key generation Alice samples a secret $x_A \in \mathbb{F}_p$ and computes the public value

$$y_A := R_{m_{i_1}^{(2)}}((\alpha^{x_A})^{i_1}) * \dots * R_{m_{i_n}^{(2)}}((\alpha^{x_A})^{i_n}), \quad (5.6)$$

where $*$ denotes entrywise matrix multiplication and $R_{(\cdot)}(\cdot)$ is the Schur–right action from (5.2). Using $m_{i_j}^{(2)} = c^{\circ(N_B B_{i_j})}$ and $R_{a \circ X}(Y) = a^{\circ(XY)}$, one obtains the compact form

$$y_A = c^{\circ(N_B \sum_{j=1}^n B_{i_j} (\alpha^{x_A})^{i_j})} =: c^{\circ(N_B Q(\alpha^{x_A}))}. \quad (5.7)$$

where Q is the (matrix–coefficient) polynomial determined by Bob’s private matrices $(B_{i_1}, \dots, B_{i_n})$:

$$Q(T) := \sum_{j=1}^n B_{i_j} T^{i_j} \in R[T], \quad \text{so that} \quad Q(\alpha^{x_A}) = \sum_{j=1}^n B_{i_j} (\alpha^{x_A})^{i_j}. \quad (5.8)$$

Here T^k denotes the usual k -fold matrix product, and the coefficients B_{i_j} multiply on the left.

Bob’s derivation From (5.7) and the Schur–left action (5.3),

$$L_{y_A}(N_B^{-1}) = L_{c^{\circ(N_B Q(\alpha^{x_A}))}}(N_B^{-1}) = c^{\circ(Q(\alpha^{x_A}))}.$$

Applying the right action (5.4) with x_B gives

$$R_{c^{\circ(Q(\alpha^{x_A}))}}(x_B) = c^{\circ(Q(\alpha^{x_A}) x_B)} =: K_B.$$

Alice’s derivation Using Bob’s public $(m_{i_j}^{(1)})$ and $R_{a \circ X}(Y) = a^{\circ(XY)}$,

$$K_A := R_{m_{i_1}^{(1)}}((\alpha^{x_A})^{i_1}) * \dots * R_{m_{i_n}^{(1)}}((\alpha^{x_A})^{i_n}) = c^{\circ(\sum_j B_{i_j} x_B (\alpha^{x_A})^{i_j})}.$$

Since $[x_B, \alpha] = 0$ implies $x_B (\alpha^{x_A})^{i_j} = (\alpha^{x_A})^{i_j} x_B$, the exponent rearranges to $Q(\alpha^{x_A}) x_B$, resulting in $K_A = K_B$.

5.2 SAA5

SAA-5 [AIJR19] is a variant of SAA-3 that slightly reorganizes the protocol so that the commutativity requirement (5.5) is no longer needed, while the essence remains the same, namely, both schemes are built on Schur-exponentiation and the associated left/right actions.

Public parameters Identical to SAA-3 and omitted: work over $R := M(d, \mathbb{F}_p)$ with index set $I = \{i_1, \dots, i_n\}$ and $V := R^n$, and reuse Schur-exponentiation $c^{\circ M}$ and the maps L_M, R_M defined above. Unlike SAA-3, SAA-5 has no global parameter α and imposes no constraint like (5.5).

Bob key generation Bob samples private

$$x_B \in R, \quad N_B \in \text{GL}_d(\mathbb{F}_p) \subset R, \quad (B_{i_1}, \dots, B_{i_n}) \in R^n, \quad c \in \mathbb{F}_p^\times.$$

(In the original instantiation, the B_{i_j} are chosen non-invertible.) He publishes

$$m_{i_j}^{(1)} := c^{\circ(B_{i_j} x_B)}, \quad m_{i_j}^{(2)} := c^{\circ(B_{i_j} N_B)} \quad (j = 1, \dots, n).$$

Alice key generation Alice chooses a private tuple $x_A = (x_{A,i_1}, \dots, x_{A,i_n}) \in R^n$ and forms the matrix-coefficient polynomial

$$Q(x_A) := \sum_{j=1}^n x_{A,i_j} B_{i_j} \in R.$$

Using Bob's $(m_{i_j}^{(2)})_j$, she computes

$$y_A := L_{m_{i_1}^{(2)}}(x_{A,i_1}) * \dots * L_{m_{i_n}^{(2)}}(x_{A,i_n}), \quad (5.9)$$

where $*$ denotes entrywise matrix multiplication and $L_{(\cdot)}(\cdot)$ is the Schur-left action from (5.1). Since $m_{i_j}^{(2)} = c^{\circ(B_{i_j} N_B)}$ and $L_{a \circ X}(Y) = a^{\circ(YX)}$, we obtain the compact form

$$y_A = c^{\circ(\sum_{j=1}^n x_{A,i_j} B_{i_j} N_B)} = c^{\circ(Q(x_A) N_B)}. \quad (5.10)$$

Bob's derivation From (5.10) and $R_{c \circ X}(Y) = c^{\circ(XY)}$,

$$R_{y_A}(N_B^{-1}) = R_{c^{\circ(Q(x_A) N_B)}}(N_B^{-1}) = c^{\circ Q(x_A)}.$$

Applying the right action with x_B gives

$$R_{c^{\circ Q(x_A)}}(x_B) = c^{\circ(Q(x_A) x_B)} =: K_B.$$

Alice's derivation From Bob's $(m_{i_j}^{(1)})_j$ and $L_{c \circ X}(Y) = c^{\circ(YX)}$,

$$K_A := L_{m_{i_1}^{(1)}}(x_{A,i_1}) * \dots * L_{m_{i_n}^{(1)}}(x_{A,i_n}) = c^{\circ(\sum_j x_{A,i_j} B_{i_j} x_B)} = c^{\circ(Q(x_A) x_B)} = K_B.$$

5.3 Attack against SAAs

In both SAA3 and SAA5 we use entrywise discrete logarithms as the inverse of Schur-exponentiation. Specifically, for a matrix $X \in R$ and a base $h \in \mathbb{F}_p^\times$, define

$$(\log_h X)_{i,j} := \log_h(X_{i,j}), \quad (5.11)$$

We abuse this notation to elements in $V := R^n$: for $X = (X_1, \dots, X_n) \in V$ set

$$\log_h X := (\log_h X_1, \dots, \log_h X_n),$$

where each component is taken entrywise as above. In addition, define a bilinear map $B : V \times V \rightarrow R$ by

$$B(X, Y) := \sum_{j=1}^n Y_j X_j \quad (X, Y \in V)$$

(Note that $B(X, Y) := \sum_{j=1}^n X_j Y_j$ in SAA5). In the notation of Section 3.3, let $g : R \rightarrow R$ be Bob's Schur map $g(X) := c^{\circ X}$ for his private base $c \in \mathbb{F}_p^\times$, and let g' be the componentwise lift defined as in (3.11). We first verify that g and B satisfy the kernel-congruence compatibility (3.14) with respect to g' in SAA3.

Let $c \in \mathbb{F}_p^\times$ and write $t := \text{ord}_p(c) \mid (p-1)$. If $g'(y) = g'(y')$ (componentwise), then for each i we have $c^{\circ y_i} = c^{\circ y'_i}$, hence

$$y_i \equiv y'_i \pmod{t} \quad \text{entrywise.} \quad (5.12)$$

Indeed, $c^u = c^v$ in $\mathbb{F}_p^\times$ iff $u \equiv v \pmod{t}$, so $c^{\circ(y_i - y'_i)} = \mathbf{1}$ (the all-ones matrix) is equivalent to (5.12).

From (5.12), for any $M \in R$ we also have

$$My_i \equiv My'_i \pmod{t} \quad \text{entrywise,}$$

because matrix multiplication is $\mathbb{Z}$ -linear in the right factor (so congruences are preserved entrywise: $(My_i)_{\ell m} = \sum_k M_{\ell k} (y_i)_{km} \equiv \sum_k M_{\ell k} (y'_i)_{km} = (My'_i)_{\ell m}$). Applying Schur-exponentiation at base c gives

$$c^{\circ(My_i)} = c^{\circ(My'_i)}.$$

This ensures the kernel-congruence compatibilities of g (3.12). In the same manner, (3.13) holds. Now, for any $x = (x_1, \dots, x_n) \in V$,

$$g(B(x, y)) = c^{\circ(\sum_{i=1}^n y_i x_i)} = \prod_{i=1}^n c^{\circ(y_i x_i)} = \prod_{i=1}^n c^{\circ(y'_i x_i)} = c^{\circ(\sum_{i=1}^n y'_i x_i)} = g(B(x, y')),$$

where we used $c^{\circ(A+B)} = (c^{\circ A}) * (c^{\circ B})$ (entrywise multiplication $*$) and the implication $y_i \equiv y'_i \pmod{t} \Rightarrow c^{\circ(y_i x_i)} = c^{\circ(y'_i x_i)}$. Therefore $g \circ B(x, \cdot)$ depends only on the tag $g'(y)$, i.e., g and B satisfy the kernel-congruence compatibility required in (3.14). The kernel-congruence compatibility also hold for g, g', B of SAA5 in the same manner.

Our goal is to show that both SAA3 and SAA5 reduce to the discrete logarithm problem (DLP). Accordingly, we assume oracle access that, for any base $h \in \mathbb{F}_p^\times$ and any $X \in R$, computes the entrywise discrete logarithm (5.11) in polynomial time. Equivalently, we establish the following as corollaries of Theorem 3.3.

Corollary 5.1 (SAA3 reduces to DLP). *Given oracle access to entrywise discrete logarithms in $\mathbb{F}_p^\times$, there is a (polynomial-time) algorithm that recovers the SSK for the SAA3 key agreement of [AR15]. In particular, breaking SAA3 is polynomial-time Turing-reducible to the DLP in $\mathbb{F}_p^\times$ (i.e., SAA3 is no harder than DLP).*

Corollary 5.2 (SAA5 reduces to DLP). *Under the same oracle access, there is a (polynomial-time) algorithm that recovers the SSK for the SAA5 key agreement of [AIJR19]. Hence breaking SAA5 is polynomial-time Turing-reducible to the DLP in $\mathbb{F}_p^\times$ (i.e., SAA5 is no harder than DLP).*

SAA3

First, following Section 3.3, set

$$x_B^{(1)} := (B_{i_1}x_B, \dots, B_{i_n}x_B) \in V, \quad x_B^{(2)} := (N_B B_{i_1}, \dots, N_B B_{i_n}) \in V.$$

The public tuples are $m^{(1)} = g'(x_B^{(1)})$ and $m^{(2)} = g'(x_B^{(2)})$, and Alice's message is

$$y_A = c^{\circ(N_B Q(\alpha^{x_A}))} \quad (\text{cf. SAA3, (5.7)}). \quad (5.13)$$

Using the oracle for (5.11) with arbitrary base h , Eve recovers the entrywise logs of the public values:

$$\log_h m_{i_j}^{(1)} = B_{i_j} x_B \log_h c, \quad j = 1, \dots, n, \quad (5.14)$$

$$\log_h m_{i_j}^{(2)} = N_B B_{i_j} \log_h c, \quad j = 1, \dots, n, \quad (5.15)$$

$$\log_h y_A = N_B Q(\alpha^{x_A}) \log_h c. \quad (5.16)$$

(All equalities are understood in $\mathbb{Z}/(p-1)\mathbb{Z}$ entrywise.) Using the information above, Eve can derive the SSK in polynomial time from *either* the Alice side or the Bob side.

From the Alice side. As explained in Section 3.1, the goal is to compute a private-key candidate x_E such that $y_A = f_A(x_E, y)$ for the relevant tag $y \in \text{Im}(g')$ (e.g., $y = m^{(2)}$ in SAA3/SAA5).

Because of the kernel-congruence compatibility, with any $z \in V$ such that $g'(z) = y$ we have the following well-defined map:

$$f_A(x, y) := R_{y_{i_1}}(x_{i_1}) * \dots * R_{y_{i_n}}(x_{i_n}) = g(B(x, z)) \quad (\forall x \in S^{(1)}).$$

Similarly, we have:

$$k_A(x, y) := R_{y_{i_1}}(x_{i_1}) * \dots * R_{y_{i_n}}(x_{i_n}) = g(B(x, z)),$$

Applying the entrywise discrete logarithm (at any fixed public base) to the defining equation for $f_A(x, y)$ reduces the recovery of x_E to solving the linear system $B(x, z) = \log(y_A)$, which is polynomial-time over $\mathbb{F}_p$ after vectorizing matrices. Thus, using (5.15) and (5.16) for any base $h \in \mathbb{F}_p^\times$, Eve solves for $x = (x_{i_1}, \dots, x_{i_n}) \in V$ under the subspace constraint $x_{i_j} \in \mathbb{F}_p[\alpha]$ the linear system

$$B(x, \log_h m^{(2)}) = \sum_{j=1}^n (\log_h m_{i_j}^{(2)}) x_{i_j} = \log_h y_A. \quad (5.17)$$

Here

$$\mathbb{F}_p[\alpha] := \left\{ \sum_{k=0}^{r-1} c_k \alpha^k : c_k \in \mathbb{F}_p \right\}, \quad r := \deg m_\alpha(T),$$

with m_α the minimal polynomial of α over $\mathbb{F}_p$. As shown in Appendix A, this constrained system has a polynomial-time solution.

Since $\log_h m_{i_j}^{(2)} = \lambda N_B B_{i_j}$ and $\log_h y_A = \lambda N_B Q(\alpha^{x_A})$ with $\lambda = \log_h c$, we obtain

$$\lambda N_B \left(\sum_{j=1}^n B_{i_j} x_{i_j} \right) = \lambda N_B Q(\alpha^{x_A}) \implies \sum_{j=1}^n B_{i_j} x_{i_j} = Q(\alpha^{x_A}),$$

because $N_B \in \text{GL}(d, \mathbb{F}_p)$ and λ is a scalar.

Using this solution (x_{i_j}) , Eve computes

$$\begin{aligned} k_A(x, m^{(1)}) &= \prod_{j=1}^n R_{m_{i_j}^{(1)}}(x_{i_j}) = \prod_{j=1}^n c^{\circ(B_{i_j} x_B x_{i_j})} = c^{\circ(\sum_{j=1}^n B_{i_j} x_B x_{i_j})} \\ &\stackrel{x_{i_j} \in \mathbb{F}_p[\alpha]}{=} c^{\circ(\sum_{j=1}^n B_{i_j} x_{i_j} x_B)} = c^{\circ(Q(\alpha^{x_A}) x_B)}, \end{aligned}$$

which is exactly the SSK. Applying the framework of Section 3.3 to the scheme of [AR15], we see that the value that effectively serves as Alice's private key is not the scalar $x_A \in \mathbb{F}_p$ chosen in [AR15], but rather the vector $(x_{i_j})_{j=1}^n \in V$ with each $x_{i_j} \in \mathbb{F}_p[\alpha]$ that satisfies (5.17). This vector is considerably easier for Eve to derive, and, contrary to the intent of [AR15], our analysis shows that the specific value of x_A itself does not play a substantive cryptographic role.

In view of the above, re-describing SAA3 within Definition 2.2, we take Alice's private-key space to be $X_A^{(1)} = V$ and the admissible subset to be $S_A^{(1)} := \{x \in X_A^{(1)} : x \in \mathbb{F}_p[\alpha]\}$; with these, the maps f_A and k_A defined above yield a PKA in that framework. The accompanying constraint $x \in \mathbb{F}_p[\alpha]$ is efficiently checkable, therefore, it does not contribute to security.

From the Bob side. Set

$$T := \{v \in V \mid [v, \alpha] = 0\}.$$

As explained in Section 3.3, Eve only needs to find $u \in V$ and $v \in T$ that satisfy the relation (3.15). Since she already knows (5.14), (5.15), she solves, for all $j = 1, \dots, n$,

$$u (\log_h m_{i_j}^{(2)}) v = \log_h m_{i_j}^{(1)}. \quad (5.18)$$

If we restrict to invertible u , set $U := u^{-1}$. Writing

$$A_j := \log_h m_{i_j}^{(2)}, \quad C_j := \log_h m_{i_j}^{(1)},$$

the system (5.18) is equivalent to the simultaneous linear equations

$$A_j v = U C_j \quad (j = 1, \dots, n), \quad (5.19)$$

together with the commutation constraint

$$v\alpha - \alpha v = 0. \quad (5.20)$$

Vectorizing with $\text{vec}(AXB) = (B^\top \otimes A) \text{vec}(X)$ yields

$$(I \otimes A_j) \text{vec}(v) - (C_j^\top \otimes I) \text{vec}(U) = 0 \quad (j = 1, \dots, n), \quad (5.21)$$

$$\iff \underbrace{\begin{bmatrix} I \otimes A_1 & -C_1^\top \otimes I \\ \vdots & \vdots \\ I \otimes A_n & -C_n^\top \otimes I \end{bmatrix}}_{\mathcal{M} \in \mathbb{F}_p^{(nd^2) \times (2d^2)}} \begin{bmatrix} \text{vec}(v) \\ \text{vec}(U) \end{bmatrix} = \mathbf{0}.$$

and (5.20) becomes

$$(I \otimes \alpha - \alpha^\top \otimes I) \text{vec}(v) = 0. \quad (5.22)$$

Stacking (5.21) over j (and appending (5.22) if needed) gives a homogeneous linear system in the unknowns $\text{vec}(v)$ and $\text{vec}(U)$. This can be solved in polynomial time over $\mathbb{F}_p$. If a solution with singular U is obtained, Eve simply selects another nonzero solution from the nullspace—an invertible solution exists (e.g. $U = N_B$) under the scheme's correctness assumptions.

By the definition $B(x, y) = \sum_{j=1}^n y_{i_j} x_{i_j}$, the R -balance of B (cf. (3.9), (3.10)) holds, since for all $x, y \in V$ and $u, v \in R$ we have

$$B(x, uy) = \sum_j (u y_{i_j}) x_{i_j} = u \sum_j y_{i_j} x_{i_j} = u B(x, y),$$

$$B(x, yv) = \sum_j y_{i_j} (x_{i_j} v) = \left(\sum_j y_{i_j} x_{i_j} \right) v = B(x, y) v.$$

Define k_E on $\text{Im}(f_A(\cdot, m^{(2)}))$ by

$$k_E(f_A(x, m^{(2)})) := R_{L_{g(B(x, s))}(u)}(v) = g(u B(x, s) v), \quad (5.23)$$

where $s \in V$ is any preimage with $g'(s) = m^{(2)}$. Then (3.18) holds automatically.

With this k_E Eve recovers the session key by applying (5.23) to y_A and using (5.7), the commutation $v\alpha = \alpha v$ from (5.20), and the relation (5.18) (equivalently $u N_B B_{i_j} v = B_{i_j} x_B$ in the SAA3 instantiation), we obtain

$$\begin{aligned} k_E(y_A) &= c^\circ u N_B Q(\alpha^{x_A}) v = c^\circ \sum_{j=1}^n u N_B B_{i_j} (\alpha^{x_A})^{i_j} v \\ &\stackrel{[v, \alpha]=0}{=} c^\circ \sum_{j=1}^n u N_B B_{i_j} v (\alpha^{x_A})^{i_j} \stackrel{(5.18)}{=} c^\circ \sum_{j=1}^n B_{i_j} x_B (\alpha^{x_A})^{i_j} \\ &\stackrel{[x_B, \alpha]=0}{=} c^\circ \sum_{j=1}^n B_{i_j} (\alpha^{x_A})^{i_j} x_B = K_A, \end{aligned}$$

SAA5 (Attack outline)

The attack on SAA5 is contained in the SAA3 attack and is even simpler, so we only explain partially. Let the bilinear form be the Frobenius (entrywise) inner product

$$B(X, Y) := \sum_j X_j Y_j \quad (X, Y \in R).$$

For this B , the left R -balance (3.9) does not hold; however, as in the SAA3 case, we do not use it in the Bob-side attack. In contrast, the right R -balance (3.10) holds for all $x, y \in V$.

Taking entrywise discrete logarithms (at any fixed base $h \in \mathbb{F}_p^\times$) one has

$$\log_h m_{i_j}^{(1)} = B_{i_j} x_B \log_h c, \quad j = 1, \dots, n, \quad (5.24)$$

$$\log_h m_{i_j}^{(2)} = B_{i_j} N_B \log_h c, \quad j = 1, \dots, n. \quad (5.25)$$

From the Alice side. Using (5.15) and $\log_h y_A$, Eve only needs to solve the linear system

$$B(x, \log_h m^{(2)}) = \sum_{j=1}^n x_{i_j} (\log_h m_{i_j}^{(2)}) = \log_h y_A, \quad (5.26)$$

for the unknowns $x = (x_{i_1}, \dots, x_{i_n})$. This is a standard linear system over $\mathbb{F}_p$ and is solvable in polynomial time; once x is obtained, the session key follows as in SAA3.

From the Bob side. Set $u = I$. Then (3.15) reduces componentwise to

$$(\log_h m_{i_j}^{(2)})v = \log_h m_{i_j}^{(1)} \quad (j = 1, \dots, n).$$

Equivalently, with $A_j := \log_h m_{i_j}^{(2)}$ and $C_j := \log_h m_{i_j}^{(1)}$,

$$\underbrace{\begin{bmatrix} A_1 \\ \vdots \\ A_n \end{bmatrix}}_{A \in \mathbb{F}_p^{nd \times d}} v = \underbrace{\begin{bmatrix} C_1 \\ \vdots \\ C_n \end{bmatrix}}_{C \in \mathbb{F}_p^{nd \times d}}.$$

This can be done in polynomial time by Gaussian elimination.

Remark 5.3. For SAA-5, Akimoto et al. [AT25] have given a polynomial-time attack. Their attack considers the following system of equations:

$$\begin{cases} \begin{pmatrix} \log_h m_{i_1}^{(1)} \\ \vdots \\ \log_h m_{i_n}^{(1)} \end{pmatrix} X = \log_h m_{i_j}^{(2)}, \\ \log_h y_A X = \log_h \kappa, \end{cases} \quad (5.27)$$

and the system

$$\begin{cases} (x_{E,i_1}, \dots, x_{E,i_n}) \begin{pmatrix} \log_h m_{i_1}^{(1)} \\ \vdots \\ \log_h m_{i_n}^{(1)} \end{pmatrix} = \log_h y_A, \\ (x_{E,i_1}, \dots, x_{E,i_n}) \begin{pmatrix} \log_h m_{i_1}^{(2)} \\ \vdots \\ \log_h m_{i_n}^{(2)} \end{pmatrix} = \log_h \kappa. \end{cases} \quad (5.28)$$

They observe that the two solutions $X \in M(d, \mathbb{Z}_p)$ and $(x_{E,i_1}, \dots, x_{E,i_n}) \in M(d \times dn, \mathbb{Z}_p)$ respectively preserve the linear relations appearing in these equations, and show that κ can be recovered uniquely as a linear combination of linearly independent vectors of the known matrices. See [AT25] for details. Whether this technique can be applied to SAA3 is not discussed there.

Remark 5.4. SAA-5 was proposed as an upgraded version of SAA-3. However, since both SAA-3 and SAA-5 fall within the general framework of Section 3.3, there is no essential difference between them. In fact, because $S^A = X_A$ (so Alice’s private key is unrestricted) and because in (3.15) one has $u = I$, the structure of SAA-5 is even simpler than that of SAA-3 from the attacker’s point of view.

Remark 5.5. an “optimised” version of SAA-5 in which Alice’s private exponents are no longer required to be short. However, this scheme still fits into the SAA-type framework of Section 3.3 and satisfies the hypotheses of Theorem 3.3. In particular, Eve can recover the shared secret key in polynomial time from the public transcript of [JIR20] without any oracle assumption on a short-exponent discrete logarithm problem.

Remark 5.6. Some of authors of SAA-5 propose a variant of SAA-5 in which the Schur exponentiation step is omitted in [JIR20]. From the SAA-type viewpoint of Section 3.3, this construction still satisfies Theorem 3.3. In particular, Eve can recover the SSK in polynomial time from the public transcript of [JIR20], without any oracle assumption on a short-exponent discrete logarithm problem.

6 Conclusion

In this paper we revisited three matrix-based asymmetric public key agreement schemes: the recent proposal of Lizama and Pino [LP24] and the schemes SAA-3 and SAA-5 of Accardi et al. [AR15, AIJR19]. Rather than analysing each protocol in isolation, we abstracted their structure in Section 3. Within this framework we identified simple algebraic conditions under which an eavesdropper can reconstruct an effective key-derivation map k_E and reduce key recovery to solving linear systems over finite fields. Our generic results for LP24-type and SAA-type schemes show that, once these conditions are met, the security of the protocol collapses to that of a linear-algebraic problem, independently of concrete parameter choices.

The concrete attacks in Sections 4 and 5 instantiate this picture. For the LP24 scheme, we gave a polynomial-time algorithm that recovers the shared secret key from public information alone by solving a single linear system derived from the underlying matrix semigroup. For SAA-3 and SAA-5, we showed that from the attacker’s perspective the two schemes fit into the same SAA-type template and fail for the same structural reason. In particular, the “updated” variants in [LP24, AIJR19] do not alter the algebraic features that enable our attacks, and hence do not provide genuine improvements in terms of resistance to key-recovery attacks. The weakness of all three schemes is therefore structural rather than parametric.

Our analysis also suggests that a “slightly improved” variant of SAA-type schemes is unlikely to restore security. As long as one remains within the current SAA template, namely, matrix rings over finite fields, a bilinear map B , and publicly known, linearly described key spaces S_A, S_B , any modest algebraic changes leave the basic linearisation phenomenon intact. By contrast, in LWE-type constructions and in Roman’kov-type noncommutative schemes, the protocol can be

viewed as being built on top of a larger solution space, with the actual PKA defined only on those solutions that lie in certain constraint sets S_A, S_B . The security intuition is precisely that, although solutions exist and are efficiently found by the honest parties, it should be computationally hard for an attacker to find any solution that belong to S_A or S_B . It would be attractive if analogous “hard” constraint sets S_A, S_B could be equipped for SAA-type or other matrix-based PKA schemes as well; our results, however, indicate that the naive choices made in existing proposals do not have this property.

A natural direction for future work is therefore to investigate, within the general framework of Section 2, for which classes of public key agreement schemes there exist constraint sets S_A, S_B that both support correct key agreement between honest parties and make it computationally infeasible for an eavesdropper to find preimages in S_A or S_B . Clarifying whether such “good” constraint sets can be constructed for noncommutative or matrix-based protocols, or whether the very structure of these schemes forbids them, remains an open problem. More broadly, our framework highlights a recurring design pattern in the literature and explains, in a single setting, why several proposals following this pattern are vulnerable. We hope that this viewpoint can also serve as a guide when formulating design criteria for future noncommutative key agreement protocols that provably avoid such linearisation attacks.

References

- [AIJR19] Luigi Accardi, Satoshi Iriyama, Koki Jimbo, and Massimo Regoli. A new class of strongly asymmetric pka algorithms: Saa-5. *Cryptography*, 3(1), 2019.
- [AIRO11] Luigi Accardi, Satoshi Iriyama, Massimo Regoli, and Masanori Ohya. Strongly asymmetric public key agreement algorithms. In *Technical Report ISEC2011-20; IEICE: Tokyo, Japan*, pages 115–121, 2011.
- [AR15] Luigi Accardi and Massimo Regoli. On a class of strongly asymmetric pka algorithms. *Journal of Mathematical Cryptology*, 9(3):151–159, 2015.
- [AT24] Genki Akimoto and Tsuyoshi Takagi. On the weak key of post-quantum key agreement saa-5. *JSIAM Letters*, 16(0):97–100, 2024.
- [AT25] Genki Akimoto and Tsuyoshi Takagi. Polynomial-time attacks on two post-quantum key agreement protocols. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences*, 2025. Advance publication.
- [BZKT18] Adi Ben-Zvi, Arkadius Kalka, and Boaz Tsaban. Cryptanalysis via algebraic spans. In Hovav Shacham and Alexandra Boldyreva, editors, *Advances in Cryptology – CRYPTO 2018*, pages 255–274, Cham, 2018. Springer International Publishing.
- [CJ03] Jung Hee Cheon and Byungheup Jun. A polynomial time algorithm for the braid diffie–hellman conjugacy problem. In Dan Boneh, editor, *Advances in Cryptology – CRYPTO 2003*, volume 2729 of *Lecture Notes in Computer Science*, pages 212–225. Springer, 2003.

- [CLM⁺18] Wouter Castryck, Tanja Lange, Chloe Martindale, Lorenz Panny, and Joost Renes. Csidh: An efficient post-quantum commutative group action. In *Advances in Cryptology – ASIACRYPT 2018: 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2–6, 2018, Proceedings, Part III*, page 395–427, Berlin, Heidelberg, 2018. Springer-Verlag.
- [DGTW19] Jintai Ding, Xinwei Gao, Tsuyoshi Takagi, and Yuntao Wang. One sample ring-lwe with rounding and its application to key exchange. In *Applied Cryptography and Network Security: 17th International Conference, ACNS 2019, Bogota, Colombia, June 5–7, 2019, Proceedings*, page 323–343, Berlin, Heidelberg, 2019. Springer-Verlag.
- [DH76] Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976.
- [DXL12] Jintai Ding, Xiang Xie, and Xiaodong Lin. A simple provably secure key exchange scheme based on the learning with errors problem. Cryptology ePrint Archive, Paper 2012/688, 2012.
- [GdKQ⁺24] Phillip Gajland, Bor de Kock, Miguel Quaresma, Giulio Malavolta, and Peter Schwabe. SWOOSH: Efficient Lattice-Based Non-Interactive key exchange. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 487–504, Philadelphia, PA, August 2024. USENIX Association.
- [JIR20] Koki Jimbo, Satoshi Iriyama, and Massimo Regoli. Implementation of a new strongly-asymmetric algorithm and its optimization. *Cryptography*, 4(3):1–19, 2020.
- [KLC⁺00] Ki Hyoung Ko, Sang Jin Lee, Jung Hee Cheon, Jae Woo Han, Ju-Sung Kang, and Choonsik Park. New public-key cryptosystem using braid groups. In Mihir Bellare, editor, *Advances in Cryptology – CRYPTO 2000*, volume 1880 of *Lecture Notes in Computer Science*, pages 166–183. Springer, 2000.
- [Kra02] Daan Krammer. Braid groups are linear. *Annals of Mathematics*, 155(1):131–156, 2002.
- [Kur06] Yalçın Kurt. A new key exchange primitive based on the triple decomposition problem. Cryptology ePrint Archive, Report 2006/378, 2006.
- [LP24] Luis Adrián Lizama-Pérez. A matrix multiplication approach to quantum-safe cryptographic systems. *Cryptography*, 8(4):56, 2024.
- [Mau93] U. M. Maurer. Secret key agreement by public discussion from common information. *IEEE Trans. Inf. Theor.*, 39(3):733–742, May 1993.
- [Pek14] Yesem Kurt Peker. A new key agreement scheme based on the triple decomposition problem. *International Journal of Network Security*, 16(6):426–436, Nov 2014.
- [Rom22] Vitaly Roman’kov. An improvement of the diffie–hellman noncommutative protocol. *Designs, Codes and Cryptography*, 90:139–153, 2022. Published online: November 14, 2021.

- [Shp08] Vladimir Shpilrain. Cryptanalysis of stickel's key exchange scheme. In *Computer Science – Theory and Applications (CSR 2008)*, volume 5010 of *Lecture Notes in Computer Science*, pages 283–288. Springer, 2008.
- [Sti05] Ernst Stickel. A new method for exchanging secret keys. In *Proceedings of the Third International Conference on Information Technology and Applications (ICITA 2005)*, pages 426–430. IEEE, 2005.

A Polynomial time solution for Alice side attack of SAA3

Proposition A.1. *Let $\alpha \in M_d(\mathbb{F}_p)$ and let $m_\alpha(T)$ be its minimal polynomial over $\mathbb{F}_p$. Then $\mathbb{F}_p[\alpha] \cong \mathbb{F}_p[T]/(m_\alpha(T))$ as $\mathbb{F}_p$ -algebras.*

Proof. By Cayley–Hamilton, the characteristic polynomial $\chi_\alpha(T) \in \mathbb{F}_p[T]$ satisfies $\chi_\alpha(\alpha) = 0$, hence the evaluation map

$$\text{ev}_\alpha : \mathbb{F}_p[T] \rightarrow M_d(\mathbb{F}_p), \quad f(T) \mapsto f(\alpha)$$

has nontrivial kernel $I := \ker(\text{ev}_\alpha) \neq \{0\}$. Clearly $\text{Im}(\text{ev}_\alpha) = \mathbb{F}_p[\alpha]$. Since $\mathbb{F}_p[T]$ is a PID, $I = (m_\alpha)$ for a unique monic polynomial m_α (the minimal polynomial of α). The first isomorphism theorem yields $\mathbb{F}_p[T]/(m_\alpha) \cong \mathbb{F}_p[\alpha]$. $\square$

Expanding each $x_{i_j} \in \mathbb{F}_p[\alpha]$ in the $\mathbb{F}_p$ -basis $1, \alpha, \dots, \alpha^{r-1}$ afforded by Proposition A.1, equation (5.17) becomes a linear system in at most nr scalar unknowns over the base field. After vectorizing matrices, this system can be solved in polynomial time by Gaussian elimination.